SEQA Project

Automated UI Testing with Python & Selenium

By Denis Denysenko, Student number: 2419040

Varna University of Management | Software Engineering Quality Assurance

Project Description

This project is an automated UI test suite for the web application Swag Labs (saucedemo.com). The goal is to verify core functionality including user authentication, logout, and product display using automated browser testing.

- Automate repetitive UI test scenarios
- Detect regressions early in the development process
- Apply Page Object Model design pattern
- Cover positive, negative, and edge case scenarios

Application Under Test

Swag Labs

saucedemo.com

A demo e-commerce web application designed specifically for UI test automation practice.

- User authentication
- Multiple user accounts
- Product inventory display
- Logout functionality

Test Users

`standard_user`

Normal access

`locked_out_user`

Blocked account

`problem_user`

UI issues

`performance_glitch_user`

Slow response

Technologies and Tools

Python 3.x

Programming language

Pytest

Test framework

Selenium WebDriver

UI automation tool

WebDriver Manager

Auto ChromeDriver install

Google Chrome

Target browser

Git / GitHub

Version control

Page Object Model (POM)

Design pattern that separates test logic from page interaction details.

Without POM

- Locators mixed with test logic
- Code duplication across tests
- Hard to maintain
- One change = many fixes

With POM ✓

- Locators in page classes
- Tests call clean methods
- Easy to maintain
- One change = one fix

Project Structure

```
seqa_project/
├── conftest.py                    # WebDriver fixture — setup & teardown
├── pages/
│   ├── login_page.py              # Page Object for Login page
│   └── inventory_page.py          # Page Object for Inventory page
└── tests/
    ├── test_login.py              # All 6 automated test cases
    └── test_data.py               # Test data separated from logic
```

Test Cases Overview

TC01 — Positive

Successful Login

TC02 — Negative

Login with Invalid Password

TC03 — Negative

Login with Empty Username

TC04 — Negative

Login with Locked Out User

TC05 — Positive

Successful Logout

TC06 — Positive

Products Displayed After Login

Authentication Tests — TC01 to TC04

TC01 Open site → Enter standard_user / secret_sauce → Click Login

Expected: Redirected to /inventory page

TC02 Open site → Enter standard_user / wrong_password → Click Login

Expected: Error: 'Username and password do not match'

TC03 Open site → Leave username empty → Enter password → Click Login

Expected: Error: 'Username is required'

TC04 Open site → Enter locked_out_user / secret_sauce → Click Login

Expected: Error: 'Sorry, this user has been locked out'

Logout & Products Tests — TC05 and TC06

TC05 — Successful Logout

1. Open saucedemo.com 2. Enter valid credentials 3. Click Login 4. Click Menu button 5. Click Logout

Expected: User is redirected back to the login page

```
assert "Swag Labs" in driver.title
```

TC06 — Products Displayed After Login

1. Open saucedemo.com 2. Enter valid credentials 3. Click Login 4. Wait for products to load 5. Collect product list

Expected: Product list is not empty (count > 0)

```
assert len(products) > 0
```

Test Results

6 / 6 Tests Passed

100% Pass Rate

TC01	Successful Login	PASSED
TC02	Login with Invalid Password	PASSED
TC03	Login with Empty Username	PASSED
TC04	Login with Locked Out User	PASSED
TC05	Successful Logout	PASSED
TC06	Products Displayed After Login	PASSED

Known Limitations

Browser Dependency

Tests are configured for Google Chrome only. Other browsers are not supported.

External Service Dependency

Tests depend on saucedemo.com availability. If the site is down, all tests will fail.

Dependencies Required

Python, Chrome, and pip packages must be installed before running the test suite.

Limited Coverage

Test suite covers authentication and inventory pages only. Other pages are not tested.

Conclusion

- Successfully implemented 6 automated UI test cases using Python, Pytest and Selenium
- Applied Page Object Model design pattern for clean, maintainable code
- Covered positive, negative and edge case scenarios
- Used explicit waits for reliable, stable test execution
- Separated test data from test logic for better maintainability

GitHub: github.com/ziperua/seqa_project